

COMMUNICATION LABORATORY

EXPERIMENT 2: Line Coding



NAME: N Deekshitha

ROLL NO: 24EC10105

GROUP 14 (Tuesday Batch)

Design and Simulation Laboratory-Communication Systems

Department of Electronics & Electrical Communication Engineering

Indian Institute of Technology Kharagpur

TITLE: Design of Line Coding Techniques

1. **PROBLEM STATEMENT:** Using the PN sequence generated in the previous experiment as data:
 1. Design and implement a circuit for the working of Unipolar Return-to-zero (Unipolar RZ) line coding scheme.
 2. Design and implement a circuit for the working of Manchester Line coding scheme.
 3. Design and implement a circuit showing the working of Alternate Mark Inversion (AMI).
 4. Implement an MLT-3 line coding technique in MATLAB for a stream of bits of digital data in software.
-

2. **KEY COMPONENTS:** • IC 7408 — Quad 2-input AND gate • IC 7432 — Quad 2-input OR gate • IC 7486 — Quad 2-input XOR gate • IC 7404 — Hex NOT gate (inverter) • IC 74164 — 8-bit parallel-out serial shift register (for PN sequence) • IC 7474 — Dual D Flip-Flop with preset and clear (for state memory / edge-triggered storage) • IC 7476 or 74LS76 — Dual JK Flip-Flop (for AMI state memory) • Operational amplifier (LM741 / TL071) — for AMI bipolar output stage • Digital Storage Oscilloscope (Tektronix TBS2104) • Function generator (for clock signal) • Breadboard and connecting wires • DC power supply (± 5 V) • Resistors (values as used in AMI op-amp circuit)

3. BASIC THEORY AND BACKGROUND

3.1 Digital Line Coding

The experiment aims at designing circuits for implementing various line coding schemes for a given digital data. Line coding techniques convert digital data to baseband digital signals, then transmit them over a channel. Line encoding schemes require certain parameters to be satisfied: minimal complexity in terms of circuitual design, transmitted power of the signal must be as low as possible, and a high tolerance level to the distortions introduced at the receiver. Also, clock synchronisation at the receiver is of paramount importance; dedicating a separate timing circuit at the transmitter and the receiver for this purpose is impractical. Hence, the timing information must be embedded along with the encoded signal such that it could be recovered at the receiver.

The purpose of line coding is therefore not merely to represent binary data electrically, but to select a waveform whose characteristics are suited to the physical transmission medium, the required data rate, and the constraints of the receiver hardware. [3][4]

3.2 Why Line Coding?

There exist numerous rationales for employing line coding. Each of the line patterns under consideration presents distinct advantages, which encompass one or more of the subsequent

benefits: [3][4]

1. **Signal Integrity:** Various distortions may be introduced into the signal due to the channel. Line coding addresses these problems by encoding digital data in a manner that reduces vulnerability to errors arising from noise and interference. Through strategic line code design, receivers can more effectively distinguish between legitimate signal transitions and erratic fluctuations. [3]
2. **Synchronization:** Line coding incorporates clock synchronization details to aid the receiver in accurately interpreting signal transition timing. This guarantees precise retrieval of the original data from the received signal. Accurate synchronization is essential to prevent data misinterpretation and to uphold communication integrity. [3][4]
3. **Efficient Spectrum Utilization:** In communication systems, optimizing bandwidth usage is crucial. Line coding aids in spreading signal energy across a wider frequency range, thus maximizing spectrum utilization. This becomes particularly significant for high-speed communication links operating within constrained bandwidth. [4][5]
4. **DC Component Removal:** Certain communication channels impose restrictions on transmitting signals with substantial DC component. Line coding methods can be employed to ensure a balanced dispersion of positive and negative voltage levels in the transmitted signal, mitigating concerns associated with DC drift (baseline wander) and bias. Baseline wander shifts the position of the signal waveform relative to the detector threshold and leads to severe erosion of noise margin. [3][4]
5. **Error Detection and Correction:** Specific line coding schemes integrate error-detection and error-correction mechanisms. These functions empower the receiver to detect and potentially rectify errors that might arise during transmission. Such capabilities are vital for preserving the precision of transmitted data, particularly over error-prone communication channels. [4][5]

3.3 Classification of Line Codes

Line coding schemes are of 5 types. They are: [3]

1. **Unipolar Line Coding:** It is analogous to a simple on-off keying scheme. It employs a single voltage level to denote binary values. Usually, one is represented by a positive or negative voltage level, while zero is indicated by the absence of a signal (no voltage or a low voltage level). Some examples include Unipolar NRZ and Unipolar RZ.
2. **Polar Line Coding:** Polar coding employs both positive and negative voltage levels to symbolize binary values, effectively handling the DC component concern inherent in unipolar coding. Some examples include Polar NRZ-L/NRZ-I, Polar RZ and Manchester coding.
3. **Bipolar Line Coding:** Bipolar coding, akin to polar coding, utilizes positive and negative voltage levels, while also incorporating zero voltage level. Such a signal is also called a duo-binary signal. Some examples include AMI (Alternate Mark Inversion) and Pseudoternary coding.
4. **Multi-transition Line Coding:** It seeks to intensify the frequency of signal transitions, aiding in clock recovery and synchronization.
5. **Multi-level Line Coding:** Multilevel coding employs more than two voltage levels to convey multiple bits simultaneously. While this can enhance data rate, it necessitates superior signal quality and can exhibit heightened susceptibility to noise. Some examples include 2B1Q (two binary, one quaternary) and 8B6T (eight binary, 6 ternary).

3.4 NRZ and RZ Sub-classification

The first three schemes can be further subclassified into non-return to zero (NRZ) and return to zero (RZ) schemes. In a non-return to zero scheme, if a signal uses positive or negative voltage to represent a binary digit, the voltage level remains the same during the whole bit period (i.e., one whole clock

period). However, in a return to zero scheme, the voltage must drop to zero in the middle of the bit period. This distinction is important: RZ schemes introduce additional transitions that aid clock recovery at the expense of increased bandwidth, while NRZ schemes maximise bandwidth efficiency at the expense of reduced timing content. [3][4]

3.5 Advantages, Disadvantages and Core Applications of Each Line Coding Scheme

Unipolar NRZ

Advantages : Simplest possible implementation; only one voltage level and a reference.

Disadvantages: Large DC component; no clock content for long runs of identical bits; highly susceptible to baseline wander.

Applications : Internal bus signaling, short-range digital logic interfaces where DC coupling and distance are not concerns. [3]

Unipolar RZ

Advantages : The mandatory return to zero within each bit period provides an additional transition that carries some timing information; easy to detect positive pulses.

Disadvantages: Requires twice the bandwidth of NRZ for the same bit rate; still has a significant DC component; clock recovery fails for long zero runs.

Applications : Laboratory demonstrations, optical communication systems (where the on/off nature maps naturally to light pulses), simple pulse-based interfaces. [3][4]

Manchester (Bi-phase)

Advantages : Guaranteed mid-bit transition in every bit period provides excellent self-clocking; zero DC component; no baseline wander; robust error detection via transition monitoring.

Disadvantages: Requires approximately twice the channel bandwidth of NRZ; higher switching activity increases electromagnetic interference.

Applications : IEEE 802.3 10BASE-T Ethernet, RFID systems, magnetic recording, serial communication in industrial and embedded systems where reliable clock recovery is more important than bandwidth efficiency. [3][9]

AMI (Alternate Mark Inversion)

Advantages : Very low DC component due to alternating polarities of marks; polarity violations provide a built-in error monitoring mechanism; spectral null at DC makes it suitable for transformer-coupled and AC-coupled channels.

Disadvantages: Long strings of zeros produce no transitions, causing clock recovery to fail; requires one bit of memory (a flip-flop) in the encoder; the receiver must distinguish three voltage levels instead of two.

Applications : T1 and E1 carrier systems in digital telephony; ISDN basic rate interface; digital subscriber line (DSL) precursor systems. B8ZS and HDB3 are practical extensions of AMI that overcome the long-zero-run problem. [4][5]

MLT-3 (Multi Level Transmit-3)

Advantages : At most one transition per bit period; the fundamental frequency of the encoded signal for a continuous stream of ones is only $1/(4T)$, reducing the required channel bandwidth by approximately a factor of two compared to NRZ; low DC content.

Disadvantages: Requires a four-state encoder (two bits of memory); three-level detection at the receiver; clock recovery is not self-contained for long zero runs; more complex decoding than binary codes.

Applications : 100BASE-TX Fast Ethernet (over Category 5 twisted-pair cable), where the reduced spectral content allows 100 Mbit/s signaling within the cable's bandwidth. MLT-3 is also a reference case for multilevel signaling in bandwidth-constrained wireline channels. [9]

In summary, the choice of line coding scheme in a communication system is governed by the specific requirements of the channel and the system: bandwidth constraints favour MLT-3; clock recovery requirements favour Manchester; DC isolation requirements favour AMI; and simplicity of implementation favours NRZ or Unipolar RZ. [3][4]

4. PN SEQUENCE USED AS INPUT DATA

The data used throughout this experiment is the PN sequence generated in Experiment 1.

A Pseudo-Noise (PN) sequence is a deterministic binary sequence generated by a Linear Feedback Shift Register (LFSR). Although deterministic, a maximal-length PN sequence has statistical properties similar to a truly random binary sequence.[3][5]

For an LFSR of degree m , the maximum period is:

$$N = 2^m - 1$$

provided the feedback polynomial is primitive and the initial state is non-zero.[5]

For a 5-stage maximal-length LFSR:

$$N = 2^5 - 1 = 31$$

The 31-bit sequence repeats periodically after every 31 clock cycles.

4.1 Primitive Polynomial

The feedback polynomial $P(x) = x^5 + x^2 + 1$ is primitive over $GF(2)$. Using initial state [1 0 0 0 1], the generated 31-bit PN sequence is:

1 0 0 0 1 1 1 1 0 0 1 1 0 1 0 0 1 0 0 0 0 1 0 1 0 1 1 1 0 1

This sequence contains 16 ones and 15 zeros — an imbalance of only one bit, characteristic of maximal-length sequences.

5. UNIPOLAR RETURN-TO-ZERO (RZ)

In Unipolar RZ coding, the signal always returns to 0 V during the second half of every bit period, regardless of the bit value. The defining properties are:

1 → +A for the first half of T_b, then 0 for the second half 0 → 0 throughout T_b

This is distinguished from Unipolar NRZ, where the signal holds at +A for the entire bit period when a 1 is transmitted. The mandatory return to zero in RZ introduces an additional transition that provides some timing information to the receiver, since a positive-going transition occurs at the start of every 1.

Mathematical representation:

For the k-th bit with value b_k:

$$x(t) = b_k \cdot A \text{ for } k \cdot T_b \leq t < k \cdot T_b + T_b/2$$

$$x(t) = 0 \text{ for } k \cdot T_b + T_b/2 \leq t < (k+1) \cdot T_b$$

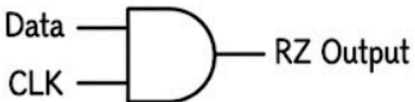
5.1 Design

The RZ encoder uses a single AND gate gated with the clock. The truth table, Boolean equation, and block diagram are as follows:

Data	CLK	RZ Output
0	0	0
1	0	0
0	1	0
1	1	1

$$\text{RZ Output} = \text{Data AND CLK}$$

$$O = D \cdot \text{CLK}$$



Return-to-Zero (RZ) Coding.

Figure D1: Unipolar RZ encoder. Truth table (left), Boolean equation (centre), and AND gate block diagram (right)

Logic relationship: RZ = D · CLK

When D=1, the clock pulse passes through the AND gate, producing a pulse for half the bit period. When D=0, the output remains zero. For a 50% duty-cycle clock at the bit rate, this produces the required return-to-zero pulse.

ICs used: IC 7408 (AND gate), IC 74164 (PN sequence generator), Function generator (clock)

5.2 Expected Outcome

Based on the encoding rule and circuit design, the following output is expected on the DSO:

- Channel 1 (PN Sequence): A two-level binary waveform switching between 0 V and +A.
- Channel 2 (RZ Output): A unipolar waveform where every binary 1 in Ch1 produces a positive pulse of amplitude +A occupying only the first half of the bit period, with the signal returning to 0 V for the remainder. Every binary 0 produces no output (0 V for the entire bit period).
- The RZ output pulse width should be exactly half the bit period ($T_b / 2$).
- No negative voltages should appear in the RZ output.
- For a PN sequence of 31 bits with 16 ones, exactly 16 positive pulses should be visible per sequence period.

5.3 Experimental Observation



Figure 1a: Unipolar RZ experimental waveform. Ch1 (yellow): PN sequence input; Ch2 (cyan): RZ encoded output



Figure 1b: Unipolar RZ waveform, observation 2



Figure 1c: Unipolar RZ waveform, observation 3

The waveforms show zero output for every binary 0 input, and a positive pulse occupying approximately one half-period for every binary 1 input, consistent with the expected Unipolar RZ encoding rule.

6. MANCHESTER LINE CODING

Manchester coding divides every bit period into two equal halves. A transition is guaranteed at the centre of every bit period, irrespective of the bit value. This property enables reliable self-clocking at the receiver.

The convention used in this experiment:

1 → +A during first half of T_b , then -A during second half (positive-to-negative transition) 0 → -A during first half of T_b , then +A during second half (negative-to-positive transition)

This means:

- A downward (positive-to-negative) mid-bit transition indicates a binary 1
- An upward (negative-to-positive) mid-bit transition indicates a binary 0
- An additional transition may occur at bit boundaries when consecutive bits differ

Mathematical representation using XOR:

For data D_k and a double-rate clock C : $S_M(t) = D_k \oplus C(t)$

The XOR with a clock at twice the bit frequency ensures the mandatory mid-bit transition.

Properties:

- No DC component — positive and negative halves of each bit cancel exactly
- Every bit contains a guaranteed mid-bit transition
- Bit boundary transitions occur only when consecutive bits are identical

6.1 Design

The Manchester encoder is realized using a single XOR gate combining the data and clock:

Data	CLK	Manchester Output
0	0	0
1	0	1
0	1	1
1	1	0

Manchester (Phase) Coding

$$\text{Manchester Output} = \text{Data XOR CLK}$$

$$O = D \oplus \text{CLK}$$

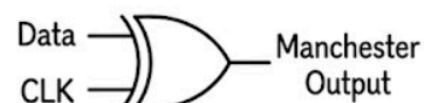


Figure D2: Manchester encoder. Truth table (left), Boolean XOR equation (centre), and XOR gate block diagram

Logic relationship: $O = D \oplus \text{CLK}$

The clock used must be at the same frequency as the bit clock, with appropriate phase alignment so that transitions occur at the correct point within the bit period.

ICs used: IC 7486 (XOR gate), IC 74164 (PN sequence generator), Function generator (clock)

6.2 Expected Outcome

Based on the XOR encoding rule, the following output is expected on the DSO:

- Channel 1 (PN Sequence): The baseband binary sequence.
- Channel 2 (Manchester Output): A bipolar waveform (swinging between +A and -A V).
- Every single bit period will have exactly one transition precisely at the centre of the bit period.
- When a binary 1 is transmitted, the signal will fall from +A to -A in the middle of the bit.
- When a binary 0 is transmitted, the signal will rise from -A to +A in the middle of the bit.
- For consecutive identical bits (e.g., 1 followed by 1), there will be an additional transition at the bit boundary to reset the level.

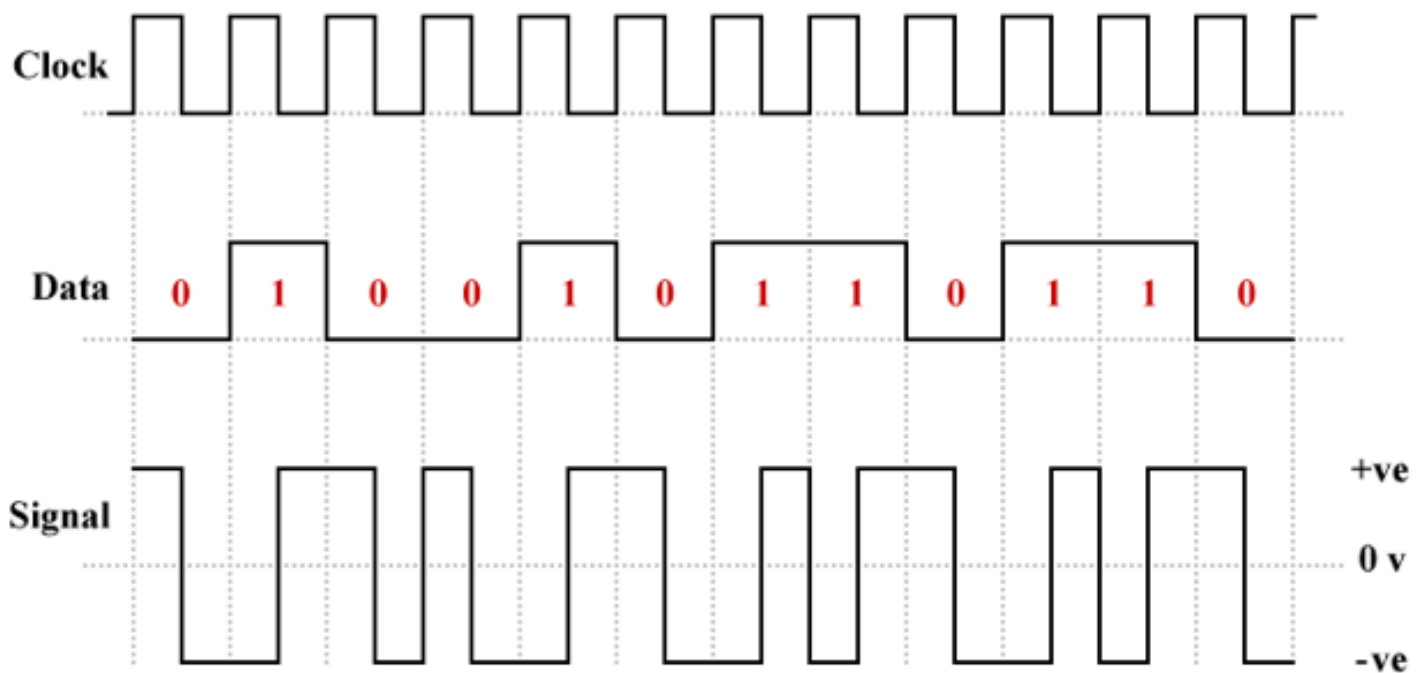


Figure 6.0: Manchester Encoding

6.3 Experimental Observation



Figure 6a: Manchester experimental waveform. Ch1 (yellow): PN sequence input; Ch2 (cyan): Manchester-encoded output



Figure 6b: Manchester waveform, observation 2. Bipolar output with guaranteed mid-bit transitions clearly visible

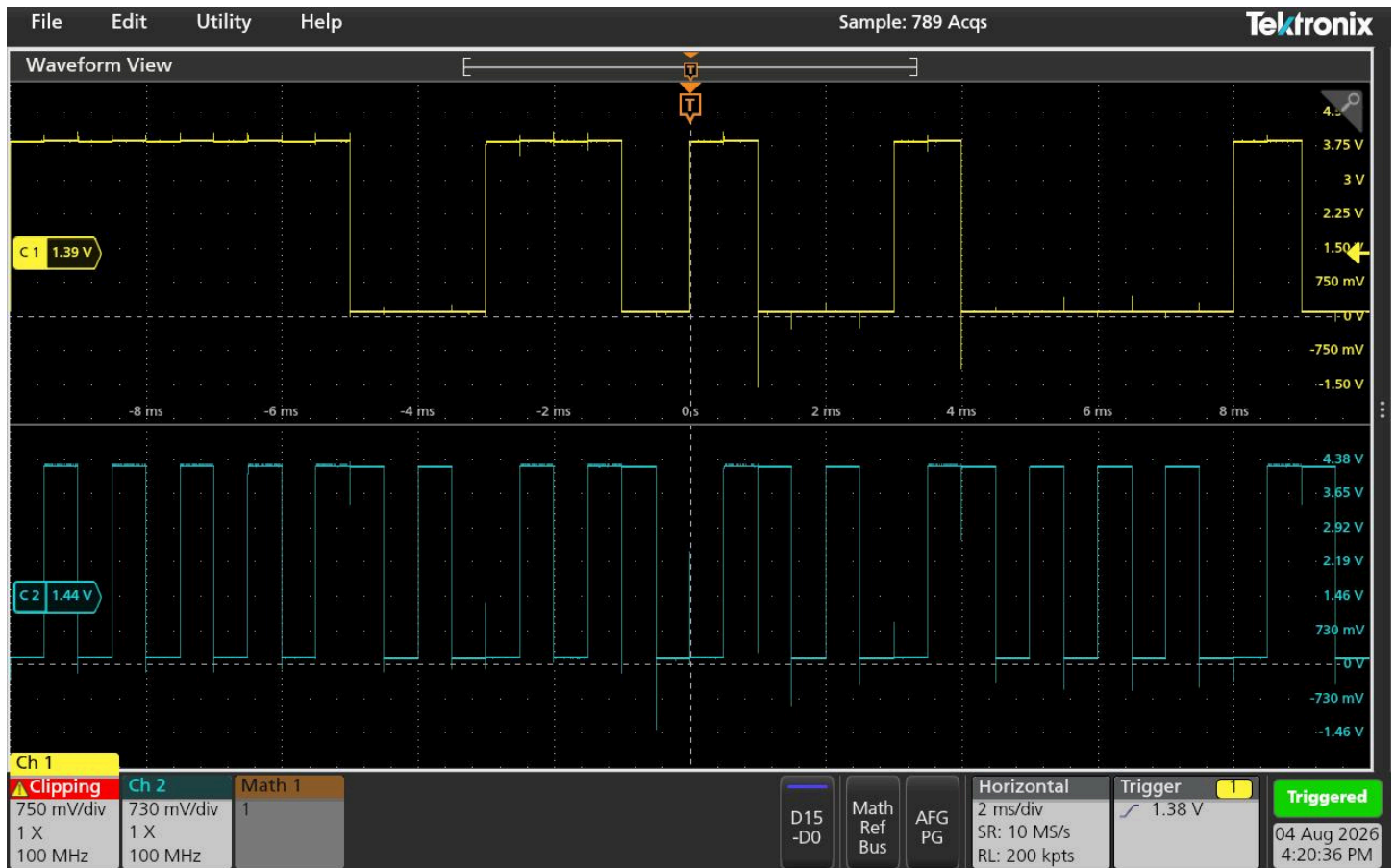


Figure 6c: Manchester waveform, observation 3

The experimental waveforms confirm that every bit period contains a mid-bit transition.

7. ALTERNATE MARK INVERSION (AMI)

AMI is a bipolar line coding scheme with three output levels: $+A$, 0 , and $-A$. The encoding rule is: [3][4]

$0 \rightarrow 0$ V Successive 1s $\rightarrow +A, -A, +A, -A, \dots$ (alternating polarity)

For example, for input sequence 1 0 1 1 0 1 (starting with positive polarity):

Output: $+A, 0, -A, +A, 0, -A$

Only binary 1s contribute to the polarity alternation. A binary 0 transmits 0 V and does NOT affect the polarity state – the state is frozen during zero bits.

Key properties:

- Three voltage levels: $+A, 0, -A$
- Average DC component ≈ 0 (positive and negative marks balance in the long run)
- Polarity violations (two consecutive marks of the same sign) indicate a transmission error
- The encoder requires memory (1 bit) – the current mark's polarity depends on previous marks

AMI is described in the addendum as a *finite-state dynamical system* with 2 states. This is formalized in the design section below.

7.1 AMI Circuit Logic and State Definition

Based on the laboratory circuit design, the AMI encoder is implemented synchronously using two D-Flip-Flops (D-FF), an XOR gate, two AND gates, and an operational amplifier. [3][4]

Circuit Architecture :

1. **Polarity State Memory (Top D-FF):** The top D-flip-flop stores the polarity state of the last transmitted mark. Its input is driven by an XOR gate: $D_1 = D_{in} \oplus Q_1$. This configuration creates a Toggle Flip-Flop (T-FF) that reverses its state (Q_1) every time the input data D_{in} is '1'. [4]
2. **Data Synchronization (Bottom D-FF):** To ensure the data aligns perfectly with the newly updated polarity state, the input data D_{in} is passed through a second D-flip-flop: $D_2 = D_{in}$. This acts as a 1-clock-cycle delay line. The synchronized data output is Q_2 (which we refer to as the synchronized data D).
3. **Mark Enable Logic (AND gates):** Two AND gates generate the unipolar pulses for the positive and negative marks by combining the synchronized data (Q_2) with the polarity state (Q_1 and $\bar{Q}_1$): $V_1 = Q_1 \cdot Q_2$ (Top AND gate) $V_2 = \bar{Q}_1 \cdot Q_2$ (Bottom AND gate)
4. **Analog Output Stage (Op-Amp):** The op-amp is configured as a difference amplifier. [3] V_1 is fed to the inverting input (-) via resistor R_1 . V_2 is fed to the non-inverting input (+) via resistor R_2 . Assuming $R_1 = R_2 = R_3 = R_f$, the output is governed by: $V_{out} = A(V_2 - V_1)$. This subtracts the V_1 component and adds the V_2 component, generating the final bipolar AMI signal. [4]

Truth Table:

Let D denote the synchronized data (Q_2), and Q denote the polarity state (Q_1). The circuit truth table perfectly matches the theoretical expectations:

D (Data)	Q (State)	$V_1 (Q \cdot D)$	$V_2 (\bar{Q} \cdot D)$	$V_{out} \propto (V_2 - V_1)$	Output Level
0	0	0	0	0	0 V
1	0	0	1	+1	+5 V
0	1	0	0	0	0 V
1	1	1	0	-1	-5 V

When $D = 0$, both AND gates are low, and V_{out} is 0 V. When $D = 1$, V_{out} alternates between +5 V and -5 V on successive ones because the state Q toggles every time D_{in} is '1'. This perfectly satisfies the AMI line coding rules. [3][4]

7.2 Hardware Implementation

The physical circuit uses the following ICs to realize the above logic:

- IC 7486: Quad 2-input XOR gate (computes $D_1 = D_{in} \oplus Q_1$). [3]

- IC 7474: Dual D-type positive-edge-triggered flip-flop (provides both the top state memory Q_1 and the bottom synchronization delay Q_2).
- IC 7408: Quad 2-input AND gate (generates V_1 and V_2).
- LM741: Operational amplifier configured as a difference amplifier to combine V_1 and V_2 into the bipolar $\pm 5V$ output. [4]

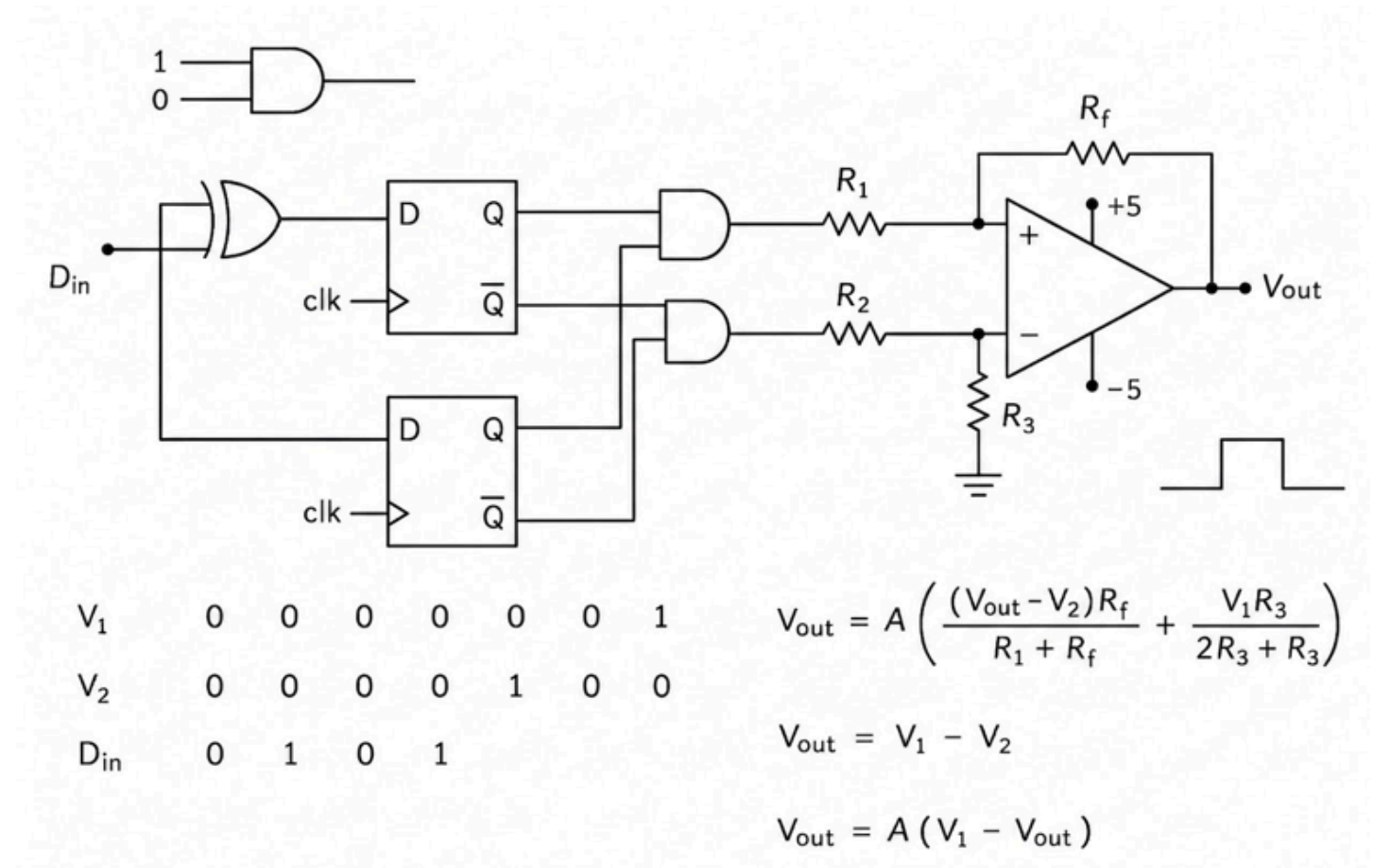


Figure D3: Schematic design of the AMI line coder using two D-flip-flops, XOR, AND logic, and an op-amp difference amplifier

7.3 Expected Outcome

Based on the encoding rules and hardware implementation, the DSO should display:

- Channel 1 (PN Sequence): The baseband binary signal.
- Channel 2 (AMI Output): A three-level waveform (+A, 0 V, -A).
- For every 0 in the PN sequence, the AMI output remains exactly at 0 V.
- For every 1 in the PN sequence, the AMI output produces a full-width pulse, but its polarity strictly alternates between positive (+A) and negative (-A) on each subsequent 1.
- Consecutive 1s (like in the sequence 1111) will produce an alternating square wave pattern: +A, -A, +A, -A.
- The peak amplitudes should symmetrically swing between approximately +3.5 V to +4 V and -3.5 V to -4 V, governed by the op-amp supply saturation.

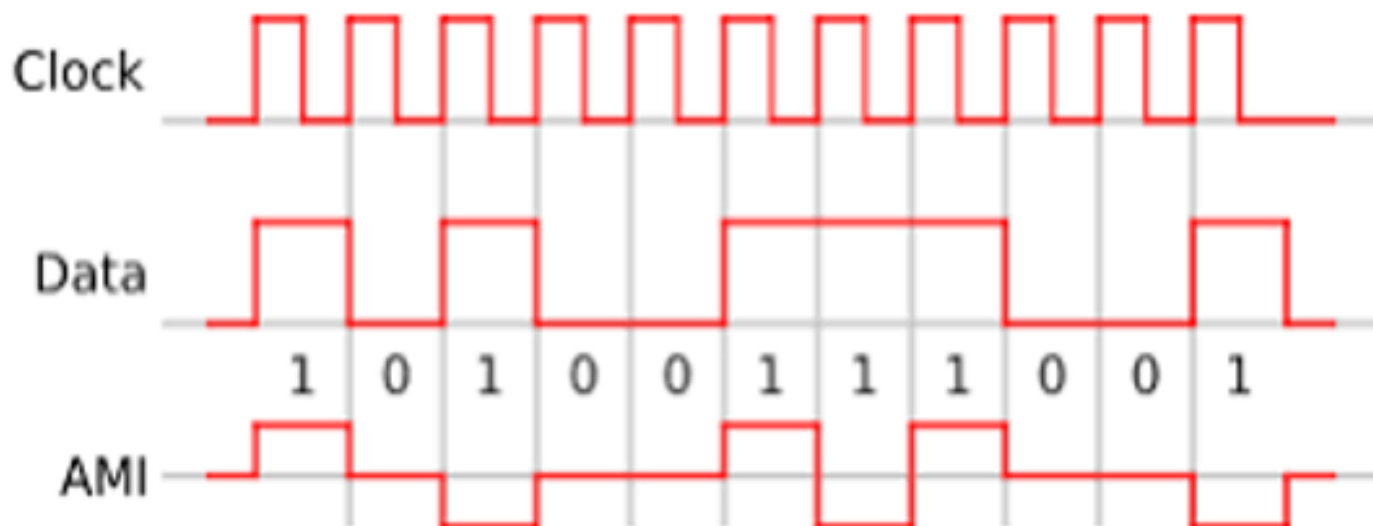


Figure 7.0: AMI Encoding

7.4 Experimental Observation



Figure 7a: AMI experimental waveform. Ch1 (yellow): PN sequence input at 1 V/div, 200 μs/div. Ch2 (cyan): AMI bipolar output at 890 mV/div, swinging between -3.56 V and +3.56 V.



Figure 7b: AMI waveform – alternating positive and negative marks clearly visible on Ch2. Input sequence on Ch1

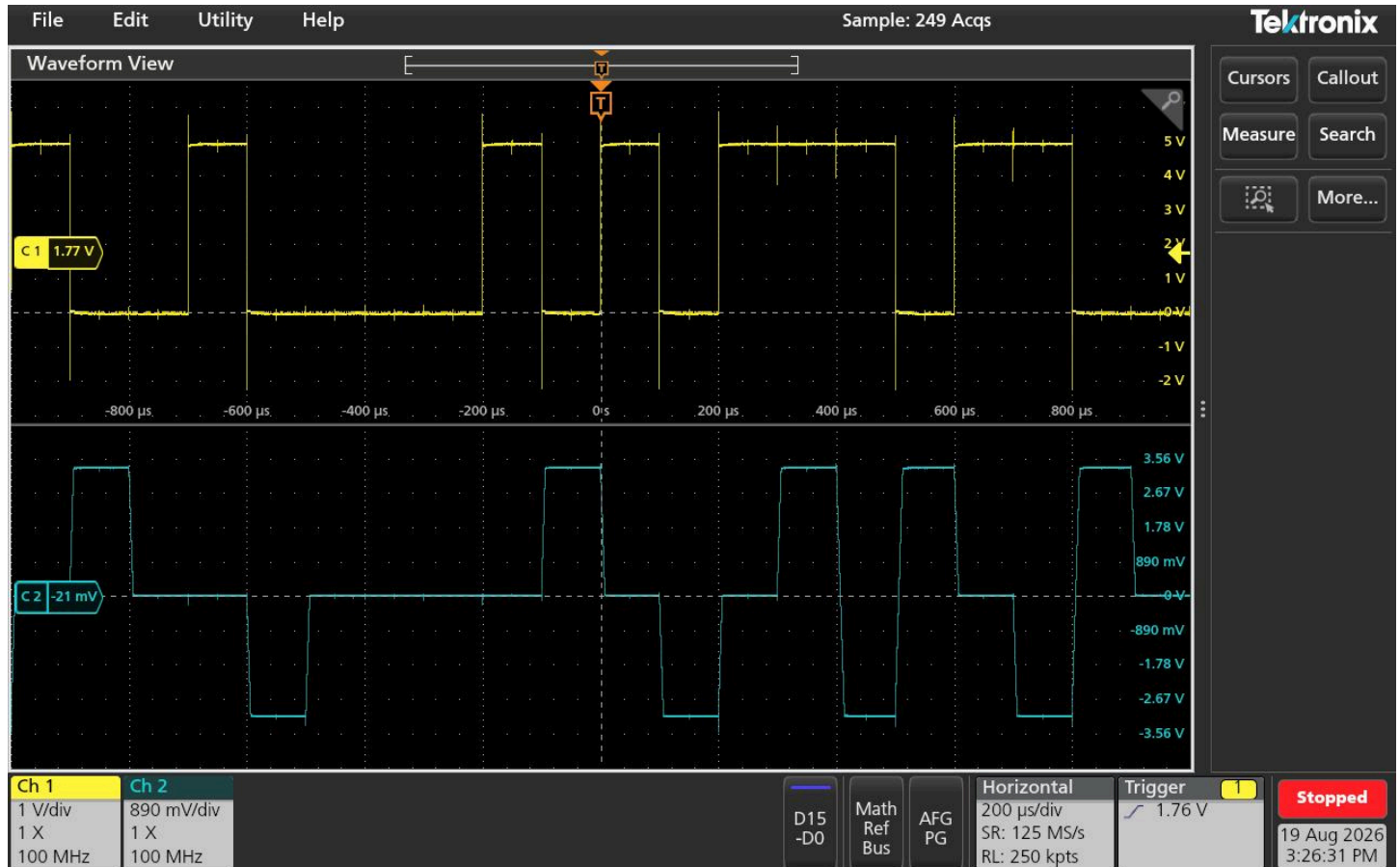


Figure 7c: AMI waveform showing the three-level nature of the output – +A ($\approx +3.56$ V), 0 V, and -A (≈ -3.56 V) are clearly distinct.



Figure 7d: AMI waveform at 400 $\mu\text{s}/\text{div}$ time scale, showing a longer portion of the PN sequence with alternating mark inversions

The experimental waveforms confirm three-level bipolar output with the alternating polarity rule. No polarity violations were observed in the recorded waveforms, indicating correct AMI encoding of the PN sequence.

8. MLT-3 LINE CODING

The MLT-3 line coding scheme follows these specific rules depending upon the current logic level:

- For any current logic level, if the next logic level is low, there is no transition in the encoded signal (i.e., if a binary 1 is followed by a binary 0, there is no transition in the signal level).
- If the current signal level is non-zero and the next bit is logic high, then the signal will transit to zero level (i.e., if a binary 1 is followed by a binary 1, then the signal level of the later binary one would transit from a signal high level to the zero level).
- If the current signal level is zero and the next bit is logic high, then the signal will transit to the level opposite to the last non-zero signal level.

8.1 MLT-3 Circuit Design (Hardware Realization)

The physical realization of MLT-3 requires tracking 4 states to cycle through the non-zero levels ($0 \rightarrow +V \rightarrow 0 \rightarrow -V$). Therefore, a Modulo-4 state machine (requiring 2 bits of memory) must be used.

(Note: Some textbook diagrams incorrectly show a Modulo-3 machine that goes $0 \rightarrow +V \rightarrow -V$, but this violates the MLT-3 rule of returning to zero between opposite polarities).

Let Q_1 and Q_0 be the outputs of two JK flip-flops forming a 2-bit synchronous counter that increments only when the input data $D = 1$.

- State 00: Output 0 V
- State 01: Output +V (Positive mark)
- State 10: Output 0 V
- State 11: Output -V (Negative mark)

JK Flip-Flop Excitation Logic (Clocked at bit rate, enabled by D):

- Flip-Flop 0 (Toggles every time $D=1$): $J_0 = D$ $K_0 = D$
- Flip-Flop 1 (Toggles when $Q_0=1$ and $D=1$): $J_1 = D \cdot Q_0$ $K_1 = D \cdot Q_0$

Output Decoder Logic:

Positive Enable (P) = $\bar{Q}_1 \cdot Q_0$ (High only in state 01)

Negative Enable (N) = $Q_1 \cdot Q_0$ (High only in state 11)

Analog Output Stage: $V_{out} = A(P - N)$ (Using a difference amplifier)

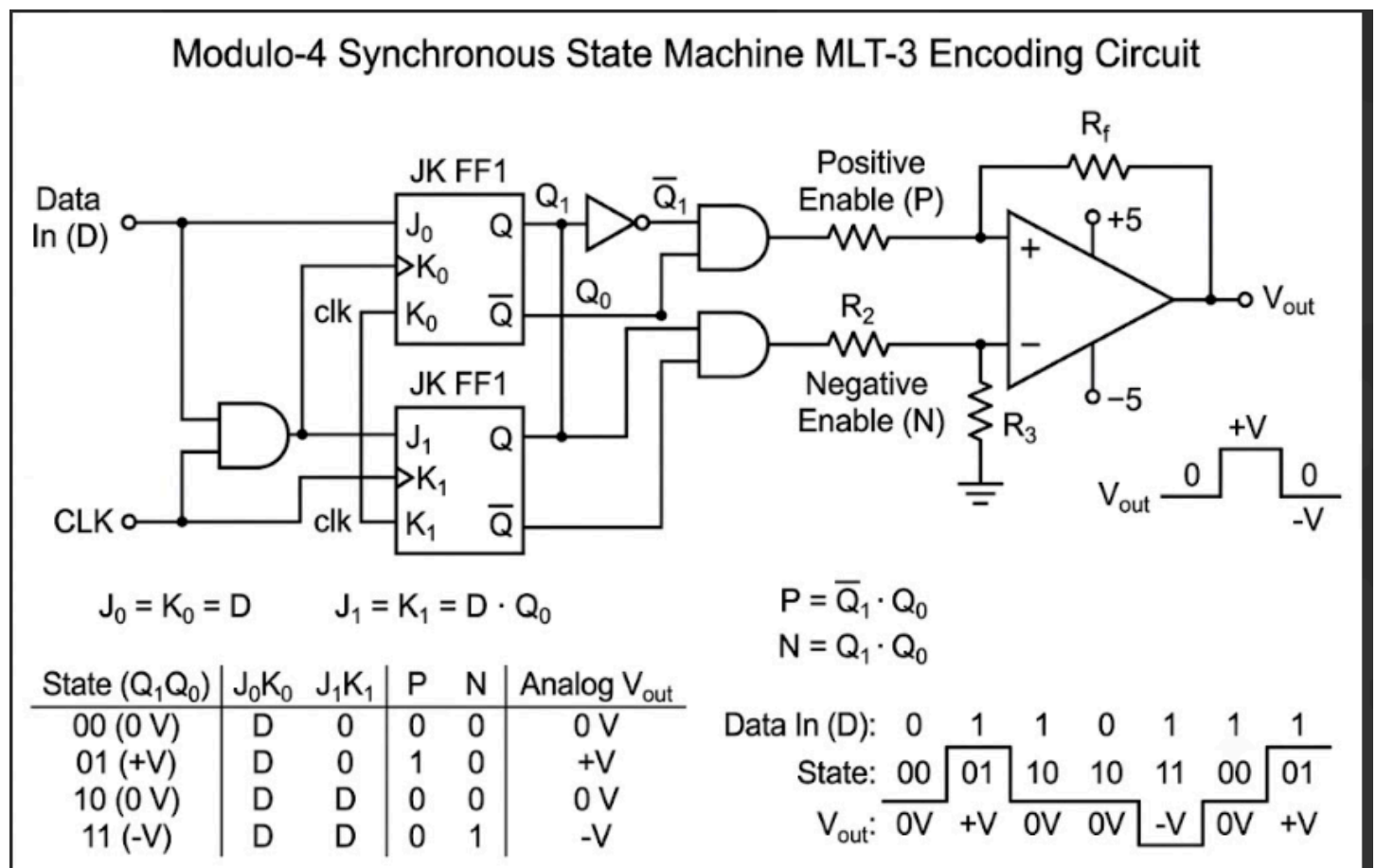


Figure D4: Schematic Diagram of MLT-3

8.2 MATLAB Code – MLT-3 Encoding

```
clc;

clear;

close all;
```

% 31-bit PN sequence (size is in the 10-100 bit range)

```
data = [1 0 0 0 1 1 1 1 1 0 0 1 1 0 1 0 ...
```

```
0 1 0 0 0 0 1 0 1 0 1 1 1 0 1];
```

```
N = length(data);
```

% MLT-3 state variables

```
level = 0;
```

```
lastNonZero = 1; % initial assumed polarity direction
```

```
mlt3 = zeros(1,N);
```

```
for k = 1:N
```

```
    if data(k) == 0
```

```
        mlt3(k) = level;
```

```
    else
```

```
        if level == 0
```

```
            level = -lastNonZero;
```

```
            lastNonZero = level;
```

```
        else
```

```
            level = 0;
```

```
        end
```

```
        mlt3(k) = level;
```

```
    end
```

```
end
```

% Plotting

```
t_data = 0:N;
```

```
data_plot = [data data(end)];
```

```
mlt3_plot = [mlt3 mlt3(end)];
```

```

figure;

subplot(2,1,1);

stairs(t_data, data_plot, 'LineWidth', 1.5);

grid on; ylim([-0.5 1.5]);

xlabel('Bit Number'); ylabel('Amplitude');

title('Input Binary PN Sequence (31 bits)');


subplot(2,1,2);

stairs(t_data, mlt3_plot, 'LineWidth', 1.5);

grid on; ylim([-1.5 1.5]);

yticks([-1 0 1]); yticklabels({'-A','0','+A'});

xlabel('Bit Number'); ylabel('Amplitude');

title('MLT-3 Encoded Signal');

```

8.3 Expected Outcome and Simulation Result

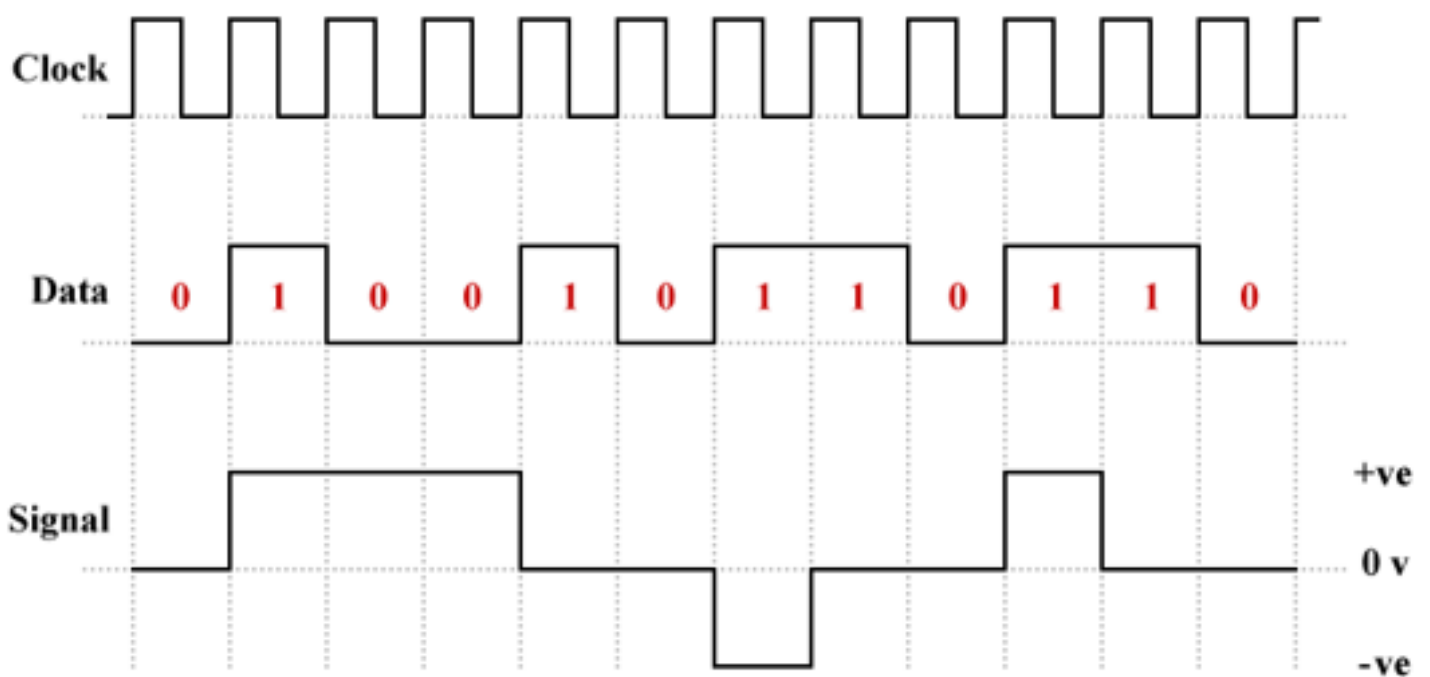


Figure 8.0: MLT-3 Encoding Scheme for binary data

When running the MATLAB code, the resulting plot explicitly proves the MLT-3 rules:

- During clusters of '0's (e.g., bits 2, 3, 4), the output holds flat (no transitions).
- During clusters of '1's (e.g., bits 5 through 9), the output clearly cycles through $0 \rightarrow +A \rightarrow 0 \rightarrow -A$.

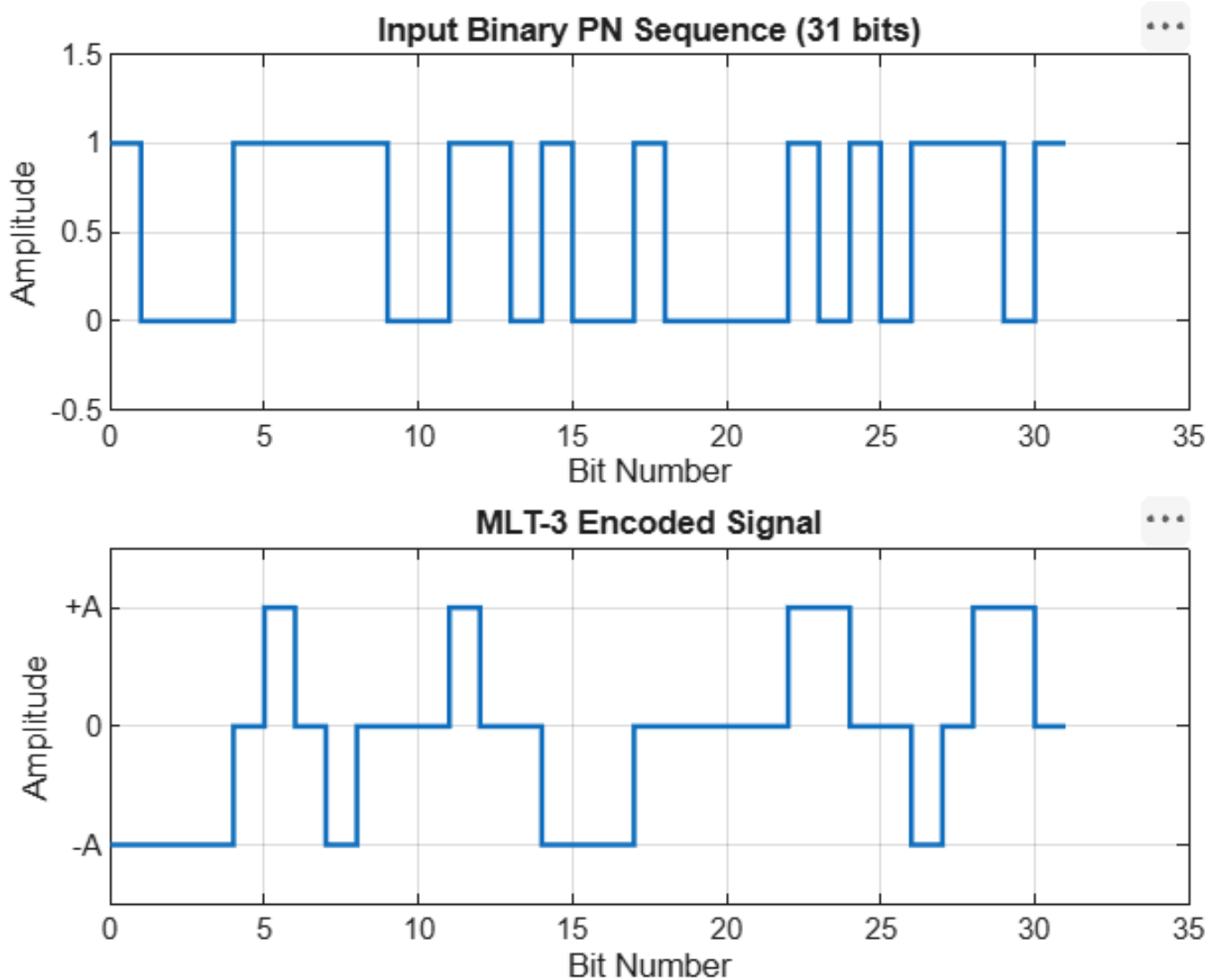


Figure 8: MATLAB simulation of MLT-3 encoding applied to the 31-bit PN sequence. The lower plot demonstrates the Modulo-4 cycling behavior and reduced transition frequency.

8.4 Uses of Multilevel Encoding Schemes (Based on Results)

As directly observed in the MATLAB simulation results (Figure 4), the MLT-3 encoded signal exhibits significantly fewer voltage transitions than the input binary data stream. While a string of consecutive '1's in the input data causes rapid high/low toggling in standard schemes, the MLT-3 signal requires four consecutive '1' bits to complete a single full cycle ($0 \rightarrow +A \rightarrow 0 \rightarrow -A$).

Because the fundamental frequency is reduced to one-fourth of the bit rate ($f = R_b / 4$), multilevel encoding dramatically improves bandwidth efficiency. This specific behavior allows high-speed data transmission over channels with severe physical bandwidth constraints—most notably, enabling 100 Mbps Ethernet (100BASE-TX) to operate over standard Category 5 copper cables rated for only ~31.25 MHz. [9]

9. DESIGN PROBLEM: EXTENSION TO 2-LEVEL AND K-LEVEL AMI

In standard "1-level AMI", there is one non-zero input state (the '1' bit) that requires 1 bit of memory to alternate its output polarity between +A and -A. Extending this to a "2-level AMI" implies the system processes a larger input alphabet that requires *two* distinct non-zero amplitude levels (e.g., A and 2A). For binary data, we achieve this by grouping the input data into dibits (2 bits at a time), which gives 4 possible input combinations.

Encoding Rule for 2-Level AMI (using Dibit Input D_1D_0):

- 00 → 0 V (No memory needed)
- 01 → Alternates between +A and -A (Requires Memory 1)
- 10 → Alternates between +2A and -2A (Requires Memory 2)
- 11 → Unused (or mapped to a synchronization violation)

9.1 Circuit Design for 2-Level AMI

Because there are two distinct non-zero levels that must alternate independently, the circuit requires two independent memory bits (Toggle Flip-Flops).

State Variables (Memory):

- Q_1 : Remembers the polarity for the '01' input (Level 1).
- Q_2 : Remembers the polarity for the '10' input (Level 2).

Next-State Logic (using D-Flip Flops acting as T-Flip Flops):

- The Level 1 memory toggles only when the input is '01': $D_{Q_1} = Q_1 \oplus (\bar{D}_1 \cdot D_0)$
- The Level 2 memory toggles only when the input is '10': $D_{Q_2} = Q_2 \oplus (D_1 \cdot \bar{D}_0)$

Output Logic:

- Level 1 Positive Enable (P_1) = $\bar{Q}_1 \cdot (\bar{D}_1 \cdot D_0)$
- Level 1 Negative Enable (N_1) = $Q_1 \cdot (\bar{D}_1 \cdot D_0)$
- Level 2 Positive Enable (P_2) = $\bar{Q}_2 \cdot (D_1 \cdot \bar{D}_0)$
- Level 2 Negative Enable (N_2) = $Q_2 \cdot (D_1 \cdot \bar{D}_0)$

Analog Output Stage (Summing difference amplifier): $V_{out} = A(P_1 - N_1) + 2A(P_2 - N_2)$

9.2 State Transition Diagram

The FSM consists of 4 global states based on the memory bits (Q_1, Q_2): {00, 01, 10, 11}. Transitions from any state (Q_1, Q_2):

- Input 00: Remain in (Q_1, Q_2), Output 0 V.

- Input 01: Toggle $Q_1 \rightarrow$ Move to $(\bar{Q}_1, Q_2)$, Output $\pm A$ based on new Q_1 .
- Input 10: Toggle $Q_2 \rightarrow$ Move to $(Q_1, \bar{Q}_2)$, Output $\pm 2A$ based on new Q_2 .

9.3 Expected Output Sequence

For an input dibit sequence: 01, 10, 01, 00, 10, 01 The output voltage levels will be:

1. 01 $\rightarrow +A$
2. 10 $\rightarrow +2A$
3. 01 $\rightarrow -A$ (alternated from $+A$)
4. 00 $\rightarrow 0\text{ V}$
5. 10 $\rightarrow -2A$ (alternated from $+2A$)
6. 01 $\rightarrow +A$ (alternated from $-A$)

9.4 Extension to k-level AMI

To extend this concept to a general k-level AMI scheme:

1. Input Alphabet: The input is grouped to form an alphabet of size $k+1$ (representing values 0 through k).
2. Amplitude Levels: The output uses $2k+1$ distinct voltage levels: $0, \pm A, \pm 2A, \dots, \pm kA$.
3. Memory Requirement: The line coder requires exactly k independent bits of memory.
4. Hardware Implementation: The circuit requires k independent Toggle Flip-Flops. Each flip-flop Q_i tracks the alternating polarity for its corresponding input level i . The final analog output is a weighted sum: $V_{\text{out}} = \sum (i \cdot A) \cdot (P_i - N_i)$ for $i = 1$ to k .

10. PN SEQUENCE AUTOCORRELATION (MATLAB)

```
clc;
```

```
clear;
```

```
close all;
```

```
pn = [1 0 0 0 1 1 1 1 1 0 0 1 1 0 1 0 0 1 0 0 0 0 1 0 1 0 1 1 1 0 1];
```

```
N = length(pn);
```

```
x = 2*pn - 1;
```

```
R = zeros(1,N);
```

```
for k = 0:N-1
```

```
    R(k+1) = sum(x .* circshift(x,[0 k])) / N;
```

```
end
```

```
lags = 0:N-1;

figure('Name', 'PN Autocorrelation', 'Position', [200 200 600 400]);

stem(lags, R, 'filled', 'LineWidth', 1.5);

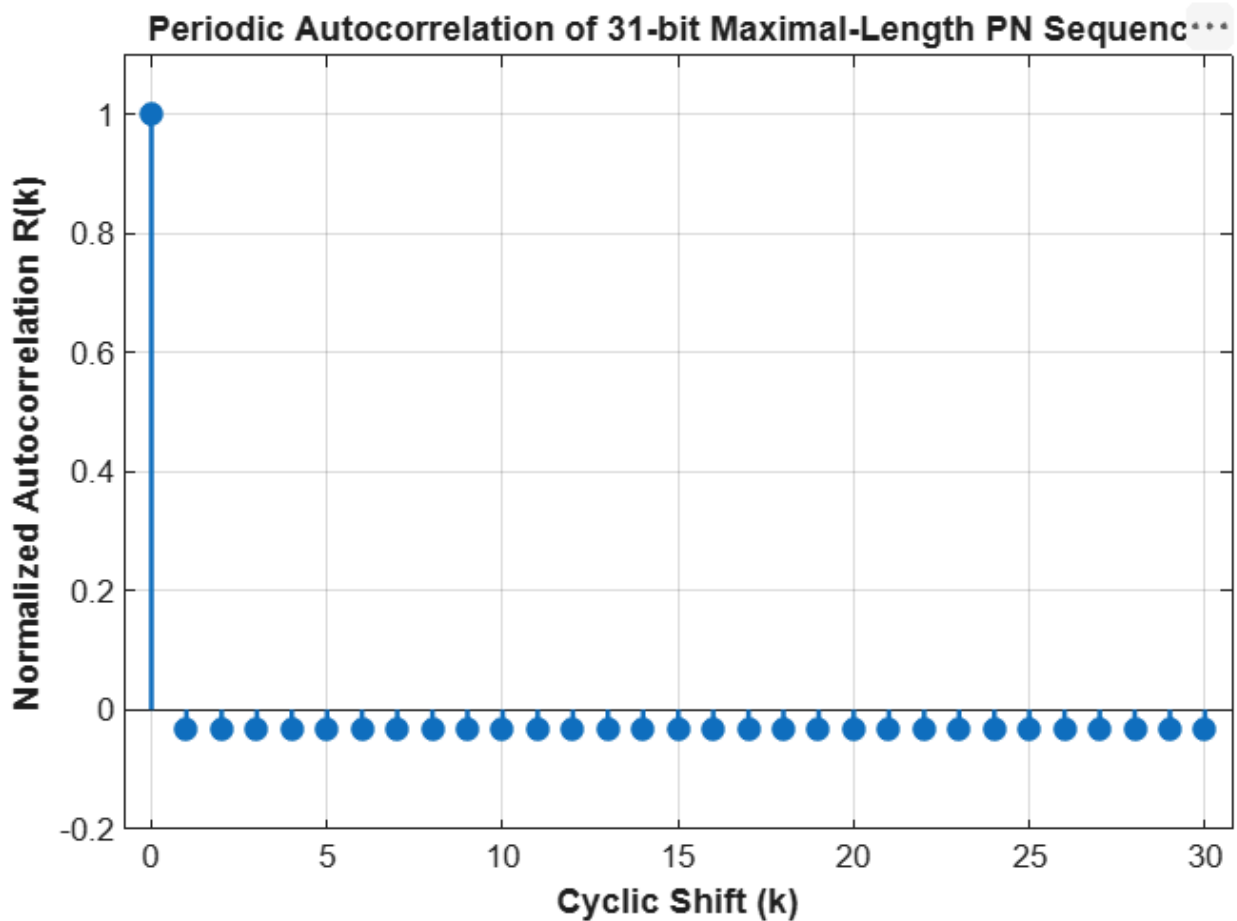
grid on;

xlabel('Cyclic Shift (k)', 'FontWeight', 'bold');

ylabel('Normalized Autocorrelation R(k)', 'FontWeight', 'bold');

title('Periodic Autocorrelation of 31-bit Maximal-Length PN Sequence');

ylim([-0.2 1.1]);
```



11. MATLAB SIMULATION OF EXPERIMENTAL WAVEFORMS

```
clc; clear; close all;

filename = 'ami.csv';
```

```
time_sec = dso_data(:, 1);

ch1_volts = dso_data(:, 2);

ch2_volts = dso_data(:, 3);

time_sec = time_sec - time_sec(1);

time_us = time_sec * 1e6;

fig = figure('Name', 'Line Coding Waveform Analysis', 'Color', 'w', 'Position', [150 150 900 600]);

ax1 = subplot(2, 1, 1);

plot(time_us, ch1_volts, 'LineWidth', 1.8, 'Color', [0.9290 0.6940 0.1250]); % Deep DSO Yellow

title('Channel 1: Baseband PN Sequence (Input)', 'FontSize', 13, 'FontWeight', 'bold');

ylabel('Amplitude (V)', 'FontSize', 12, 'FontWeight', 'bold');

grid on; grid minor;

ax1.YLim = [min(ch1_volts)-1, max(ch1_volts)+1];

ax2 = subplot(2, 1, 2);

plot(time_us, ch2_volts, 'LineWidth', 1.8, 'Color', [0.3010 0.7450 0.9330]); % Bright DSO Cyan

title('Channel 2: Line Coded Output Signal', 'FontSize', 13, 'FontWeight', 'bold');

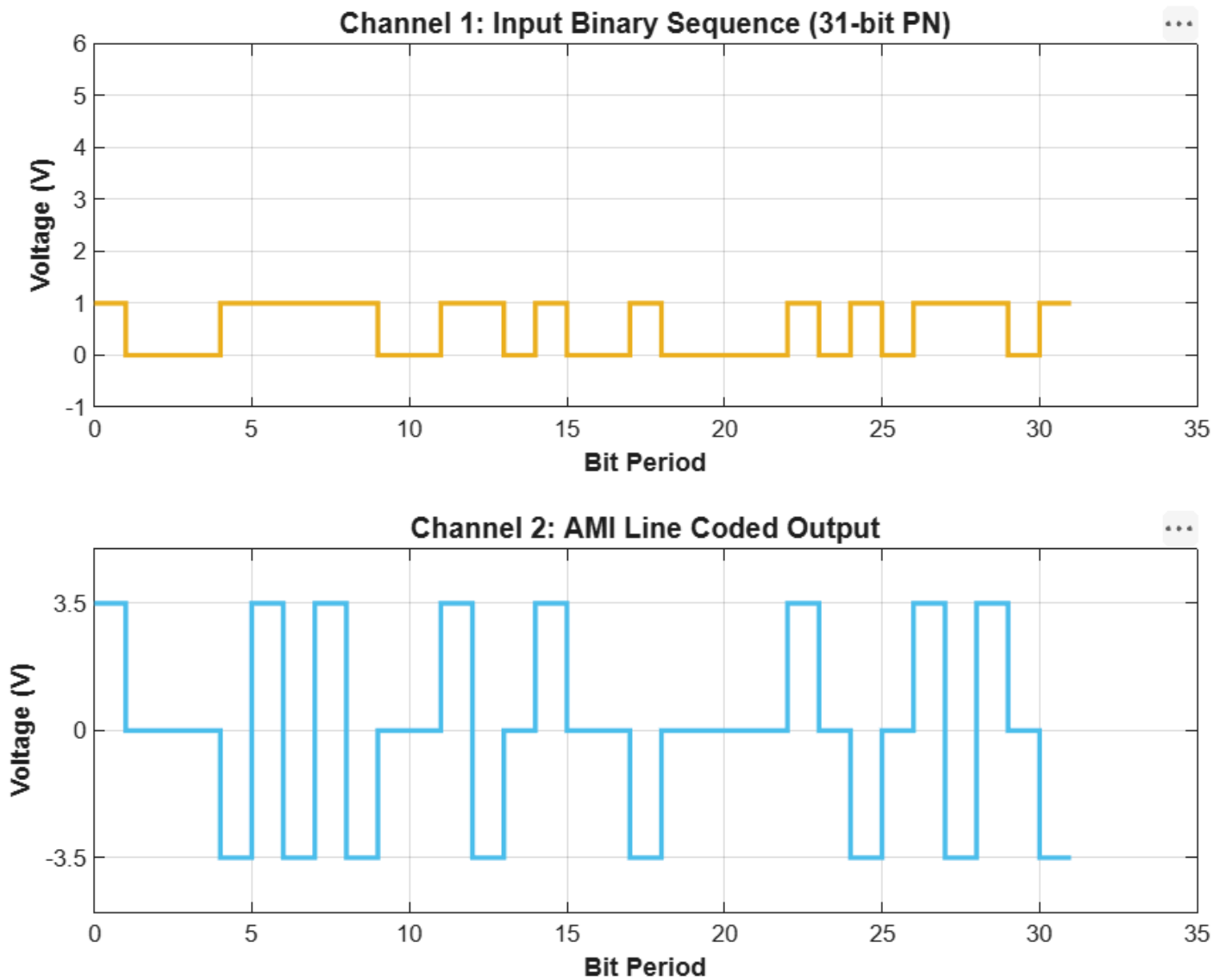
xlabel('Relative Time (\mu s)', 'FontSize', 12, 'FontWeight', 'bold');

ylabel('Amplitude (V)', 'FontSize', 12, 'FontWeight', 'bold');

grid on; grid minor;

ax2.YLim = [min(ch2_volts)-1, max(ch2_volts)+1];

linkaxes([ax1, ax2], 'x');
```



12. DISCUSSION AND COMPARISON OF LINE CODING TECHNIQUES

A brief discussion of the four line coding schemes explored:

Unipolar RZ: The simplest scheme. Binary 1 produces a half-period positive pulse; binary 0 produces nothing. Simple to generate (one AND gate) but has a significant DC component and requires more bandwidth than NRZ because of the mandatory zero-return transition. Clock recovery is poor for long zero runs since no transitions occur.

Manchester: A self-synchronizing polar scheme with a guaranteed mid-bit transition in every bit period. Excellent clock recovery properties. Zero DC component – the positive and negative halves of each bit cancel exactly. The price is approximately twice the bandwidth of NRZ. Preferred in short-to-medium range systems where timing reliability is paramount.

AMI: A three-level bipolar scheme with 1 bit of encoder memory. The alternating-mark rule keeps the DC component near zero and introduces a rudimentary error detection capability (polarity violations). The trade-off is poor clock recovery during long zero runs. As a finite-state dynamical system (2 states, period-2 polarity cycle), AMI is the simplest stateful line code.

MLT-3: A three-level scheme with 4 encoder states (period-4 cycle). Inserts a zero level between every pair of nonzero marks, reducing the maximum frequency component in the transmitted spectrum by approximately a factor of 2 compared to AMI or NRZ. The key metric is bandwidth efficiency: MLT-3 allows 100 Mbit/s signaling over Cat-5 cable by keeping the fundamental frequency at 25 MHz (vs. 100 MHz for NRZ). Used in 100BASE-TX.

Summary table:

Parameter	Unipolar RZ	Manchester	AMI	MLT-3
Voltage levels	0, +A	+A, -A	+A, 0, -A	+A, 0, -A
DC component	High	Zero	Very low	Low
Synchronization	Moderate	Excellent	Moderate	Moderate
Bandwidth (rel. to NRZ)	2×	2×	~NRZ	~NRZ/2
Encoder memory	None	None	1 bit (2 states)	2 bits (4 states)
Error detection	No	No	Polarity violation	No
Typical use	Lab demos	Ethernet (10BASE-T)	T1/E1 telephony	100BASE-TX

13. INFERENCES

1. Different line coding schemes represent identical binary data using substantially different waveform structures, each with distinct spectral and synchronization properties.
2. Unipolar RZ is simple to implement but suffers from a significant DC component and poor clock recovery for long zero sequences.
3. Manchester coding provides excellent self-synchronization through its guaranteed mid-bit transition, but approximately doubles the required channel bandwidth.
4. AMI significantly reduces the DC component through alternating mark polarity, and provides a rudimentary error detection mechanism via polarity violation checks.
5. AMI is the simplest stateful line coder — its encoder is a 2-state finite-state machine with a T-flip-flop (JK FF with $J=K=D$) as the only memory element.
6. The JK flip-flop conversion shows directly that AMI's state update $Q^* = Q \oplus D$ maps cleanly to $J=D$ and $K=D$, confirming the T-FF toggle behavior.

7. MLT-3 extends AMI's key idea to a 4-state machine, inserting zero levels between nonzero marks. This halves the maximum spectral frequency compared to NRZ or AMI, making it ideal for bandwidth-constrained transmission (e.g., 100BASE-TX Ethernet).
8. Both AMI and MLT-3 are finite-state dynamical systems: $s_{k+1} = f(s_k, b_k)$. Their valid transmitted waveforms form constrained subsets of signal space, compactly represented as paths through a trellis.
9. Optimal decoding of stateful codes (AMI, MLT-3) naturally leads to the Viterbi/Bellman recursion — a discrete dynamic programming algorithm structurally analogous to the Hamilton-Jacobi-Bellman equation of optimal control.
10. A 5-stage maximal-length LFSR with primitive polynomial x^5+x^2+1 generates a 31-bit PN sequence with near-ideal autocorrelation: $R(0)=1$ and $R(k)\approx-1/31$ for $k\neq 0$.
11. Experimental waveforms closely match theoretical predictions. Minor discrepancies arise from propagation delays, finite rise/fall times, op-amp bandwidth, and oscilloscope sampling artifacts.

14. CONCLUSION

This experiment successfully demonstrated the design, implementation, and analysis of four digital line coding techniques using a 31-bit PN sequence as the data source.

Unipolar RZ, Manchester, and AMI were implemented in hardware. The experimental oscilloscope waveforms confirm correct operation: the RZ waveform shows half-period positive pulses, the Manchester waveform shows guaranteed mid-bit transitions, and the AMI waveform shows a clean three-level bipolar output with alternating mark polarity.

The complete AMI design was derived from first principles — from the FSM state definition and state transition table, through JK flip-flop conversion ($J=K=D$), to the op-amp output stage. The derivation confirms that AMI is fundamentally a 2-state finite-state dynamical system whose state update is a simple XOR operation.

The design problem establishes that MLT-3 extends AMI's 1-level key ideas to a 4-state machine with a period-4 cyclic output, achieving superior bandwidth efficiency at the cost of additional encoder complexity. The dynamical systems perspective — formalized in the addendum — unifies both schemes under the framework $s_{k+1}=f(s_k, b_k)$, connecting line coding through trellis decoding to Bellman's dynamic programming and ultimately to optimal control.

15. REFERENCES AND BIBLIOGRAPHY

- [1] Experiment 2 Line Coding Lab Manual
- [2] Experiment 2 Line Coding Lab Report given for reference
- [3] S. Haykin and M. Moher, Communication Systems, 5th ed., John Wiley & Sons, 2010.
- [4] B. P. Lathi and Z. Ding, Modern Digital and Analog Communication Systems, 4th ed., Oxford University Press, 2009.
- [5] J. G. Proakis and M. Salehi, Digital Communications, 5th ed., McGraw-Hill, 2008.

[6] R. E. Bellman, Dynamic Programming, Princeton University Press, 1957.

[7] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," IEEE Trans. Inf. Theory, vol. 13, no. 2, pp. 260–269, Apr. 1967.

[8] MathWorks, MATLAB R2023a Documentation — Signal Processing Toolbox.

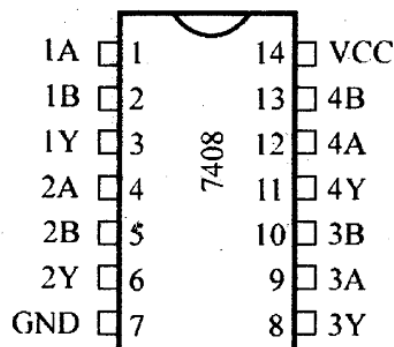
[9] IEEE Std 802.3-2018, Section 25 — 100BASE-TX physical layer specification (MLT-3).

APPENDIX A — EQUIPMENT LIST AND PIN DIAGRAMS

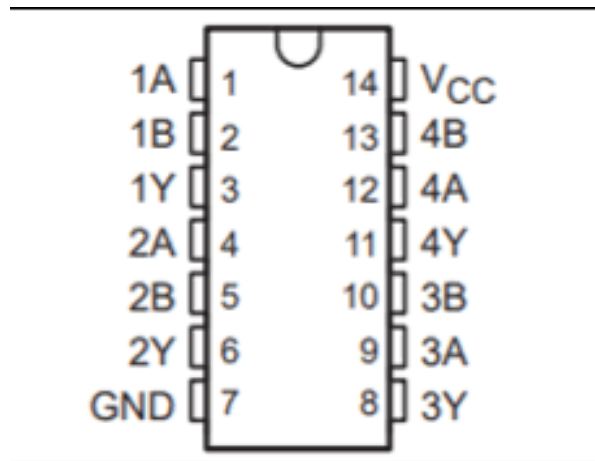
A.1 Equipment Used

1. IC 7408 — Quad 2-input AND gate (positive logic)
2. IC 7432 — Quad 2-input OR gate (positive logic)
3. IC 7486 — Quad 2-input XOR gate (positive logic)
4. IC 7404 — Hex inverter (NOT gate)
5. IC 74164 — 8-bit serial-in, parallel-out shift register (PN sequence generator)
6. IC 7474 — Dual D Flip-Flop with preset and clear (edge-triggered state storage)
7. IC 7476 (or 74LS76) — Dual JK flip-flop with preset and clear (AMI state memory)
8. Operational amplifier — LM741 or TL071 (AMI bipolar output stage)
9. Tektronix TBS2104 Digital Storage Oscilloscope
10. Function generator (bit clock source)
11. Breadboard and connecting wires
12. DC power supply (± 5 V for ICs; op-amp supply as required)
13. Resistors

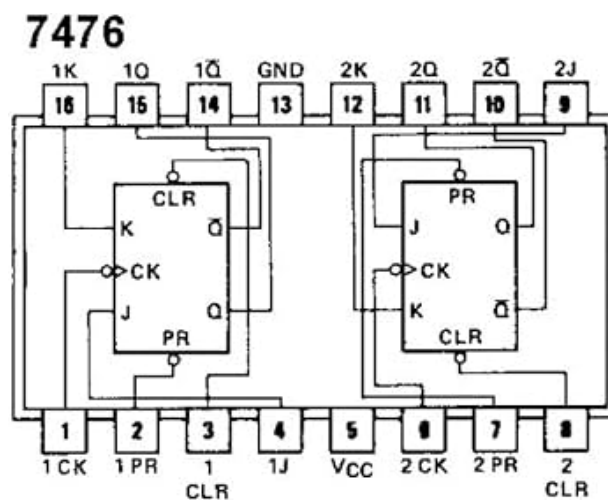
A.2 Pin Diagrams



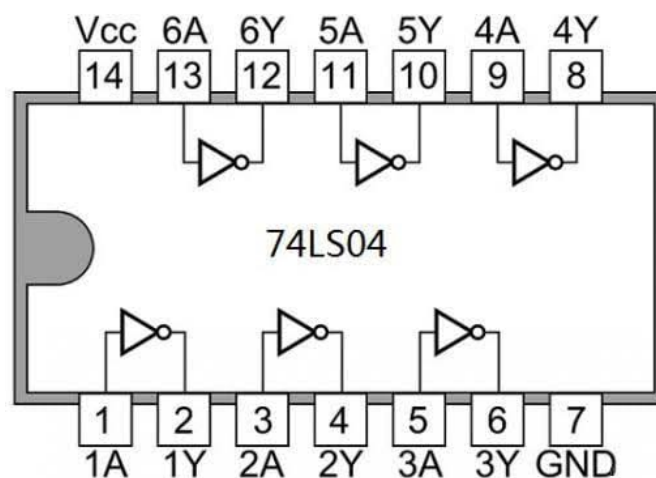
IC 7408 pin diagram — 14-pin DIP. Label VCC (pin 14), GND (pin 7). Gates: 1A-1B-1Y (pins 1,2,3), 2A-2B-2Y (pins 4,5,6), 3A-3B-3Y (pins 8,9,10), 4A-4B-4Y (pins 11,12,13). Pins used in this experiment: Gate 1 for $P=D \cdot \bar{Q}$, Gate 2 for $N=D \cdot Q$



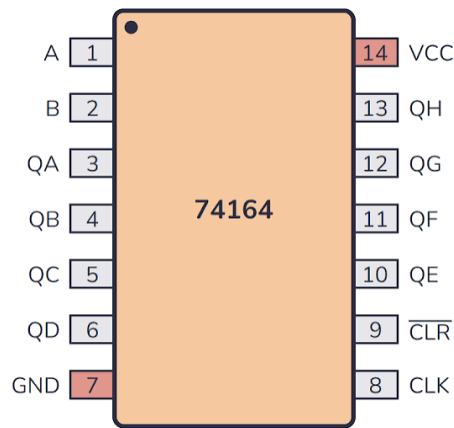
IC 7486 pin diagram – 14-pin DIP. Label VCC (pin 14), GND (pin 7). Gates: 1A-1B-1Y (pins 1,2,3), 2A-2B-2Y (pins 4,5,6), 3A-3B-3Y (pins 8,9,10), 4A-4B-4Y (pins 11,12,13). Pins used: Gate 1 for Manchester encoder $D \oplus CLK$



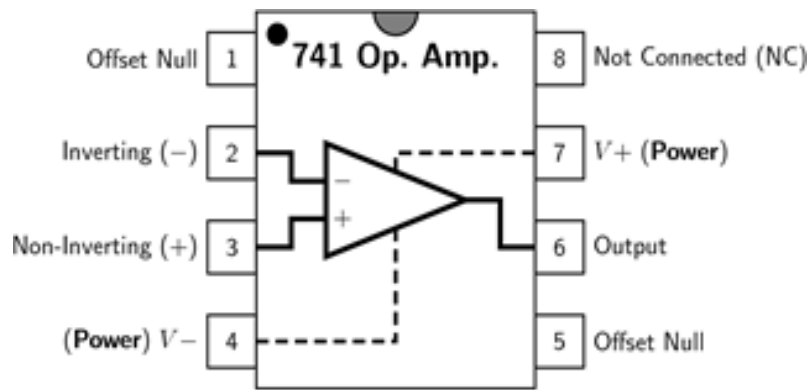
IC 7476 (JK FF) pin diagram – 16-pin DIP. Key pins: 1J, 1CLK, 1K, 1PRE, 1CLR, 1Q, 1 $\bar{Q}$ (pins 14,1,2,13,15,12,11). VCC pin 5, GND pin 13. Used for AML state memory: J=D, K=D, clock=bit clock, Q=state output



IC 7404 pin diagram – 14-pin DIP. Label VCC (pin 14), GND (pin 7). Six inverters: 1A-1Y (pins 1,2), 2A-2Y (pins 3,4), etc. Pins used: one inverter for $\bar{Q}$ in positive mark logic $P = D \cdot \bar{Q}$



IC 74164 pin diagram – 14-pin DIP. Key pins: A, B (serial inputs, pins 1,2), QA–QH (parallel outputs, pins 3,4,5,6,10,11,12,13), CLK (pin 8), CLR (pin 9), VCC (pin 14), GND (pin 7). Used as 5-stage LFSR for PN sequence generation



Op-amp (LM741 or TL071) pin diagram – 8-pin DIP. Key pins: IN- (pin 2), IN+ (pin 3), V+ (pin 7), V- (pin 4), OUT (pin 6). Configured as difference amplifier: $V_{out} = A(P - N)$ for AMI bipolar output stage